Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Develop Blockchain and Web3 App with Vibe Coding

TGS Ref No: TGS-2021009338

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version v18.1

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
17.0	Original source	Original trainer slides and approved assessment content	Tertiary Infotech Academy
v18.0	13 September 2026	Technical rebuild, 12 executable activities and visual source mapping	Tertiary Infotech Academy
v18.1	13 September 2026	Kept legacy procedures in notes/LG; concept-led slides and aligned final release	Tertiary Infotech Academy

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

How to use this Learner Guide

This guide covers Develop Blockchain and Web3 App with Vibe Coding (TGS-2021009338), v18.1. It includes all 12 self-contained Solidity and Python Web3 activities, detailed setup, public testnet deployment instructions and the original v17 content for traceability. Python is the client and tester; compiled Solidity runs on an actual EVM test chain. The slide deck shows mechanisms, prompts and evidence; this guide contains the detailed procedure.

Course identity and learning outcomes

LO1: Analyze business requirements for Blockchain, Cryptocurrency, Smart Contract and Token Economics.

LO2: Code smart contract for Ethereum blockchain using Solidity programming.

LO3: Test smart contract and monitor its outputs.

LO4: Deploy smart contract and ensure coding documentation is accurate.

TSC Programming and Coding FSE-DIT-4018-1.1. Topic1: A1 K1 K2; Topic2: A2 A3 K3; Topic3: A4 A5 K4; Topic4: A6 A7 K5. Training is two days and 16 programme hours (13.5 training and 2.5 assessment). Approved assessment instruments are WA 60 minutes and PP 90 minutes, within the 16-hour programme. The authenticated course record and original approved plan allocate13.5 training hours plus2.5 assessment hours in the16-hour programme.

Download and package verification

1. Open https://lms-tms.tertiaryinfotech.com and sign in using the credentials issued by the provider. 2. Open the course named Develop Blockchain and Web3 App with Vibe Coding, TGS-2021009338. 3. Open the current courseware link. 4. Download the learner slide PDF, Learner Guide and Activities. 5. Confirm all covers carry v18.1 and the correct code. 6. Keep each activity folder intact; do not move run_activity.py without Contract.sol, contract.json, compile.cjs and mock records. Contact the trainer if a link or identity differs.

Test chain and toolchain boundaries

Baseline executed toolchain: Solidity/solc-js 0.8.30; optimizer enabled with 200 runs; Cancun EVM; OpenZeppelin Contracts 5.4.0; web3.py 8.0.0; eth-tester 0.14.0b1; Py-EVM 0.12.1b1. Every folder pins its dependencies. Install Python 3.12/3.13 and Node.js 22 LTS. The default EthereumTesterProvider is an actual in-process EVM chain. It is distinct from public Sepolia, does not exercise consensus/finality and cannot be accessed by a browser wallet. Use Anvil localhost for the optional browser route. Current dependencies should be reviewed before production; these examples are reproducible educational contracts.

Topic 1 Blockchain, Cryptocurrency, Smart Contract and Token Economics

Mapped evidence: A1, K1, K2.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 01 Hash and anchor certificate evidence

Goal

register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance.

Activity workflow

Synthetic hash-evidence record	HashRegistry actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 65–69. Folder: labs/activity-01-hash-evidence.

register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance.

Mapped learning evidence: A1, K1, K2. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace HashRegistry against the acceptance requirement above. For ERC contracts, inspect the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Digest registration exact execution sequence

Read the synthetic certificate record. Canonicalize JSON with sorted keys and compact separators, hash with SHA-256 and call register(digest) as the issuer. Read registered(digest) before/after. Attempt the identical digest as issuer and a forged digest as outsider. Both calls must revert.

Expected observable state: registered(digest)=true; duplicate and unauthorized writes rejected.

Edit and verify task

Change one record field and prove its digest differs; register the revised evidence as issuer. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build hash and anchor certificate evidence for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: register(digest) commits a SHA-256 evidence digest; issuer authorization and duplicate rejection define acceptance. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract HashRegistry {
 address public immutable issuer=msg.sender;
 mapping(bytes32=>bool) public registered;
 event Registered(bytes32 indexed digest,address indexed issuer);
 function register(bytes32 digest) external {
  require(msg.sender==issuer,"issuer only"); require(!registered[digest],"duplicate");
  registered[digest]=true; emit Registered(digest,msg.sender);
 }
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.register(digest));assert
contract.functions.registered(digest).call();reject(contract.functions.register(digest),a);reject(contract.functions.register(Web3.k
eccak(text="forged")),c)
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 02 Map buyer requirements to Ether escrow

Goal

Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release.

Activity workflow

Synthetic	Escrow actor/state	Real EVM transaction	Python assertion and

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 70–74. Folder: labs/activity-02-requirements-escrow.

Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release.

Mapped learning evidence: A1, K1, K2. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace Escrow against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Buyer seller escrow exact execution sequence

Read amount_eth from the mock record and convert using to_wei. Constructor seller=account1, buyer=account0. fund() with price minus1 wei must revert; fund exact price succeeds. Outsider release fails; buyer release increases seller balance by exactly price; second release fails.

Expected observable state: funded=true; released=true; escrow balance zero; seller gains0.1ETH.

Edit and verify task

Add a delivery evidence digest requirement before buyer release, and add a failing test before editing Solidity. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build map buyer requirements to ether escrow for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Created → funded → released; exactly 0.1 ETH is paid once and only the buyer authorizes release. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract Escrow {
 address public immutable buyer; address payable public immutable seller;
 uint public immutable price; bool public funded; bool public released;
 constructor(address payable s,uint p){buyer=msg.sender;seller=s;price=p;}
 function fund() external payable {require(msg.sender==buyer &&
!funded,"buyer/state");require(msg.value==price,"price");funded=true;}
 function release() external {require(msg.sender==buyer && funded && !released,"buyer/state");released=true;(bool
ok,)=seller.call{value:price}("");require(ok,"payment");}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
price=w3.to_wei(mock["amount_eth"],"ether");contract=deploy(ctor=(b,price));reject(contract.functions.fund(),a,price-1);tx(contr
act.functions.fund(),a,price);reject(contract.functions.release(),c);before=w3.eth.get_balance(b);tx(contract.functions.release());
assert w3.eth.get_balance(b)-before==price;reject(contract.functions.release(),a)
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Topic 2 Ethereum Blockchain and Solidity Programming

Mapped evidence: A2, A3, K3.

Activity 03 Compile and review a supplier purchase order

Goal

enum state, immutable quantity and supplier-only transitions implement a business requirement trace.

Activity workflow

Synthetic solidity-order record	PurchaseOrder actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

separately evidenced.

Slide reference: 153–157. Folder: labs/activity-03-solidity-order.

enum state, immutable quantity and supplier-only transitions implement a business requirement trace.

Mapped learning evidence: A2, A3, K3. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace PurchaseOrder against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Supplier transition trace exact execution sequence

Read quantity from mock data; constructor stores supplier account1, quantity and hashed record ID. Attempt accept as account2, then accept as supplier. Read state=1. Deliver as supplier, read state=2 and reject repeated deliver.

Expected observable state: quantity=25; state Created0→Accepted1→Delivered2; outsider/repeated calls fail.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Edit and verify task

Add buyer confirmation as state3 with buyer-only transition and test supplier cannot confirm. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build compile and review a supplier purchase order for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enum state, immutable quantity and supplier-only transitions implement a business requirement trace. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract PurchaseOrder {
 enum State {Created,Accepted,Delivered} State public state;
 address public immutable buyer=msg.sender; address public supplier;
 uint public immutable quantity; bytes32 public immutable orderId;
 constructor(address s,uint q,bytes32 id){require(q>0,"quantity");supplier=s;quantity=q;orderId=id;}
 function accept() external {require(msg.sender==supplier && state==State.Created,"supplier/state");state=State.Accepted;}
 function deliver() external {require(msg.sender==supplier && state==State.Accepted,"supplier/state");state=State.Delivered;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy(ctor=(b,int(mock["quantity"]),Web3.keccak(text=mock["record_id"])));reject(contract.functions.accept(),c);tx(co
ntract.functions.accept(),b);assert contract.functions.state().call()==1;tx(contract.functions.deliver(),b);assert
contract.functions.state().call()==2;reject(contract.functions.deliver(),b)
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 04 Issue and transfer ERC-20 workshop credits

Goal

ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units.

Activity workflow

Synthetic erc20-credits record	WorkshopToken actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 158–162. Folder: labs/activity-04-erc20-credits.

ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units.

Mapped learning evidence: A2, A3, K3. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace WorkshopToken against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

ERC20 integer units exact execution sequence

Read token_transfer=500 and token_allowance=20 from mocks; multiply by10**decimals. Transfer issuer→learner, then approve spender from learner and transferFrom learner→spender. Query balances, allowance and totalSupply. Reject an overdraft from learner.

Expected observable state: Issuer999500 WSC; learner480; spender20; totalSupply1000000 WSC.

Edit and verify task

Change allowance to10 and assert transferFrom20 fails before spending10. Explain smallest-unit arithmetic. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Original PP full balance transfer

1. Deploy a fresh WorkshopToken instance to isolate the original all-token transfer requirement. 2. Read balanceOf(issuer) and totalSupply: both are1000000×10^18 smallest units. 3. As issuer call

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

transfer(learner,all_tokens), wait for receipt status1. 4. Read issuer balance0, learner balance all_tokens and unchanged totalSupply. 5. Reject a further1unit transfer from the now-empty issuer. 6. Capture before/after balances, receipt and explanation. The earlier500/20 allowance exercise uses a separate instance and is not substituted for this full-balance test.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build issue and transfer erc-20 workshop credits for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: ERC-20 balanceOf, transfer, approve and transferFrom preserve totalSupply with 18-decimal integer units. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract WorkshopToken is ERC20 {constructor() ERC20("Workshop Credits","WSC")
{_mint(msg.sender,1000000*10**decimals());}}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
 if "execution reverted" in str(exc).lower(): return
 raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();unit=10**18;supply=contract.functions.totalSupply().call();tx(contract.functions.transfer(b,int(mock["token_tra
nsfer"])*unit));assert
contract.functions.balanceOf(b).call()==int(mock["token_transfer"])*unit;tx(contract.functions.approve(c,int(mock["token_allowa
nce"])*unit),b);tx(contract.functions.transferFrom(b,c,int(mock["token_allowance"])*unit),c);assert
contract.functions.balanceOf(c).call()==int(mock["token_allowance"])*unit;assert
contract.functions.totalSupply().call()==supply;reject(contract.functions.transfer(a,501*unit),b)
# Original PP full-balance transfer evidence on a fresh token deployment.
full_token=deploy();all_tokens=full_token.functions.balanceOf(a).call()
tx(full_token.functions.transfer(b,all_tokens));assert full_token.functions.balanceOf(a).call()==0
assert full_token.functions.balanceOf(b).call()==all_tokens
assert full_token.functions.totalSupply().call()==all_tokens
reject(full_token.functions.transfer(c,1),a)

manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 05 Issue and revoke ERC-721 course certificates

Goal

ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status.

Activity workflow

Synthetic certifynft record	CertificateNFT actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 163–167. Folder: labs/activity-05-certifynft.

ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Mapped learning evidence: A2, A3, K3. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace CertificateNFT against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Certificate verification exact execution sequence

Canonicalize mock evidence and issue certificate ID1 to account1 as issuer. Read ownerOf, documentHash and supportsInterface(0x80ac58cd). Attempt issue as outsider. Revoke ID1 as issuer and read revoked=true; ownership remains account1.

Expected observable state: Owner account1; digest equals canonical SHA256; ERC721 interface true; revoked true.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Edit and verify task

Add a verify function returning holder+digest+not-revoked, and test revoked credentials return false. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build issue and revoke erc-721 course certificates for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: ERC-721 ownerOf and documentHash prove holder and document binding; revocation remains an explicit status. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
contract CertificateNFT is ERC721 {
 address public immutable issuer=msg.sender;
 mapping(uint=>bytes32) public documentHash; mapping(uint=>bool) public revoked;
 constructor() ERC721("Course Certificate","CERT"){}
 function issue(address learner,uint id,bytes32 digest) external
{require(msg.sender==issuer,"issuer");documentHash[id]=digest;_safeMint(learner,id);}
 function revoke(uint id) external {require(msg.sender==issuer,"issuer");require(_ownerOf(id)!
=address(0),"missing");revoked[id]=true;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.issue(b,1,digest));assert contract.functions.ownerOf(1).call()==b;assert
bytes(contract.functions.documentHash(1).call())==digest;reject(contract.functions.issue(c,2,digest),c);tx(contract.functions.rev
oke(1));assert contract.functions.revoked(1).call();assert
contract.functions.supportsInterface(bytes.fromhex("80ac58cd")).call()
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 06 Track custody for pharmaceutical batches

Goal

Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts.

Activity workflow

Synthetic supplyverify record	SupplyVerify actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 168–172. Folder: labs/activity-06-supplyverify.

Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts.

Mapped learning evidence: A2, A3, K3. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace SupplyVerify against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Pharmaceutical custody exact execution sequence

Read mock batch identifier BATCH-SG-042 and hash it. Create batch as manufacturer account0 with canonical evidence digest. Reject outsider handover; handover manufacturer→distributor account1 then distributor→retailer account2. Read batches(id) after each state change. Reject duplicate create.

Expected observable state: custodian account2; stage2; digest unchanged; wrong-custodian and duplicate create fail.

Edit and verify task

Add temperature evidence as a new digest and a permitted rejection/quarantine state with exact role tests. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build track custody for pharmaceutical batches for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Batch ID + evidence digest + custodian + stage create a custody state machine; unauthorized handover reverts. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract SupplyVerify {
 struct Batch {address custodian;bytes32 digest;uint stage;} mapping(bytes32=>Batch) public batches;
 event Handover(bytes32 indexed id,address indexed from,address indexed to,uint stage);
 function create(bytes32 id,bytes32 digest) external
{require(batches[id].custodian==address(0),"exists");batches[id]=Batch(msg.sender,digest,0);}
 function handover(bytes32 id,address to) external {Batch storage b=batches[id];require(b.custodian==msg.sender && to!
=address(0),"custodian");address old=b.custodian;b.custodian=to;b.stage++;emit Handover(id,old,to,b.stage);}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
 raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();id=Web3.keccak(text=mock["batch"]);tx(contract.functions.create(id,digest));reject(contract.functions.handover(id,b),c);tx(contract.functions.handover(id,b));assert contract.functions.batches(id).call()[0]==b;tx(contract.functions.handover(id,c),b);assert contract.functions.batches(id).call()[2]==2;reject(contract.functions.create(id,digest),a)
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art["evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.keccak(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"backend":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"assertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Topic 3 Smart Contract Testing

Mapped evidence: A4, A5, K4.

Activity 07 Test eligible voting and duplicate controls

Goal

enrol defines eligibility; vote bounds option and flips voted before incrementing count.

Activity workflow

Synthetic ballot-tests record	Ballot actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 207–211. Folder: labs/activity-07-ballot-tests.

enrol defines eligibility; vote bounds option and flips voted before incrementing count.

Mapped learning evidence: A4, A5, K4. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace Ballot against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Eligibility and ballot test matrix exact execution sequence

Chair account0 enrols account1. Account2 vote must fail. Account1 vote option2 must fail (only0/1). Account1 vote1 succeeds, votes(1)=1 and voted(account1)=true. Repeat vote1 fails.

Expected observable state: votes(1)=1; votes(0)=0; ineligible, invalid-option and duplicate cases rejected.

Edit and verify task

Add voting deadline and test votes fail after tester time travel beyond it. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build test eligible voting and duplicate controls for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enrol defines

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

eligibility; vote bounds option and flips voted before incrementing count. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract Ballot {
 address public immutable chair=msg.sender; mapping(address=>bool) public eligible; mapping(address=>bool) public voted;
mapping(uint=>uint) public votes;
 function enrol(address a) external {require(msg.sender==chair,"chair");eligible[a]=true;}
 function vote(uint option) external {require(eligible[msg.sender] && !
voted[msg.sender],"eligibility");require(option<2,"option");voted[msg.sender]=true;votes[option]++;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.enrol(b));reject(contract.functions.vote(0),c);reject(contract.functions.vote(2),b);tx(contract.functions.vote(1),b);assert contract.functions.votes(1).call()==1;reject(contract.functions.vote(1),b)
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art["evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.keccak(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"backend":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"assertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 08 Verify withdrawal controls against reentrancy

Goal

Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary.

Activity workflow

Synthetic reentrancy-review record	SafeVault actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 212–216. Folder: labs/activity-08-reentrancy-review.

Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary.

Mapped learning evidence: A4, A5, K4. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace SafeVault against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Adversarial receive callback exact execution sequence

Depositor account0 deposits1ETH. Deploy ReenterProbe with vault address. Account1 funds attack0.1ETH; receive callback attempts vault.withdraw recursively and records blocked=true. Read probe balance entry0 and vault balance1ETH. Withdraw original depositor; vault ends zero.

Expected observable state: ReenterProbe.blocked=true; victim deposit preserved1ETH; no duplicate payout.

Edit and verify task

Temporarily remove guard and zeroing in a separate vulnerable copy, demonstrate theft only in this isolated chain, then restore controls. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build verify withdrawal controls against reentrancy for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract SafeVault {
 mapping(address=>uint) public balance; bool private entered;
 function deposit() external payable {balance[msg.sender]+=msg.value;}
 function withdraw() external {require(!entered,"reentrant");uint
value=balance[msg.sender];require(value>0,"empty");entered=true;balance[msg.sender]=0;(bool
ok,)=msg.sender.call{value:value}("");require(ok,"send");entered=false;}
}
contract ReenterProbe {
 SafeVault public vault; bool public blocked;
 constructor(address v){vault=SafeVault(v);}
 function attack() external payable {vault.deposit{value:msg.value}();vault.withdraw();}
 receive() external payable {try vault.withdraw() {} catch {blocked=true;}}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.deposit(),a,w3.to_wei("1","ether"));probe=deploy("ReenterProbe",
(contract.address,));tx(probe.functions.attack(),b,w3.to_wei(mock["amount_eth"],"ether"));assert
probe.functions.blocked().call();assert contract.functions.balance(probe.address).call()==0;assert
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

w3.eth.get_balance(contract.address)==w3.to_wei("1","ether");tx(contract.functions.withdraw());assert w3.eth.get_balance(contract.address)==0
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art["evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.keccak(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"backend":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"assertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 09 Measure gas for storage transition classes

Goal

Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price.

Activity workflow

Synthetic gas-monitoring record	GasBench actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 217–221. Folder: labs/activity-09-gas-monitoring.

Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price.

Mapped learning evidence: A4, A5, K4. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace GasBench against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Measured storage gas exact execution sequence

Write lookup key0 from zero→42, then42→43. Append42 to array. Capture each receipt.gasUsed and compare first>second. Read lookup(0)=43 and count=1. Inspect gas-results.csv from this locked run.

Expected observable state: first SSTORE gas greater than overwrite; key0 value43; one array entry; CSV includes measured values.

Edit and verify task

Repeat the same write43→43 and add a measured no-op row; do not assume the compiler/effect cost before measuring. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build measure gas for storage transition classes for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Measure cold zero-to-nonzero versus nonzero-to-nonzero SSTORE through receipts; report measured gas, not an assumed universal price. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract GasBench {
 uint256[] public values; mapping(uint=>uint) public lookup;
 function append(uint value) external {values.push(value);}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
 function write(uint key,uint value) external {lookup[key]=value;}
 function count() external view returns(uint){return values.length;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();first=tx(contract.functions.write(0,42)).gasUsed;second=tx(contract.functions.write(0,43)).gasUsed;append=t
x(contract.functions.append(42)).gasUsed;assert contract.functions.lookup(0).call()==43;assert first>second;(BASE/"gas-
results.csv").write_text(f"operation,gas_used\nzero_to_nonzero,{first}\nnonzero_to_nonzero,{second}\narray_append,
{append}\n")
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 10 Index receipt events and reconcile records

Goal

Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list.

Activity workflow

Synthetic event-indexing record	EventLedger actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 222–226. Folder: labs/activity-10-event-indexing.

Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list.

Mapped learning evidence: A4, A5, K4. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace EventLedger against the acceptance requirement above. For ERC contracts, inspect the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Indexed evidence reconciliation exact execution sequence

Read synthetic record_id, hash as bytes32 and record it with canonical digest. Wait for receipt then decode Evidence event. Require one log, matching digest and recorded(id)=true. Reject duplicate record. Inspect indexed-events.json and reconcile its block and digest to mock input.

Expected observable state: One decoded event; exact input digest; on-chain recorded true; duplicate false attempt.

Edit and verify task

Add a second mock record, index both events in block order and flag a tampered local digest. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build index receipt events and reconcile records for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Event ID, digest and timestamp are decoded from actual EVM receipts and reconciled with the business evidence list. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
contract EventLedger {
 address public immutable operator=msg.sender;
 mapping(bytes32=>bool) public recorded;
 event Evidence(bytes32 indexed id,bytes32 digest,uint timestamp);
 function record(bytes32 id,bytes32 digest) external {require(msg.sender==operator &&
!recorded[id],"operator/duplicate");recorded[id]=true;emit Evidence(id,digest,block.timestamp);}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();id=Web3.keccak(text=mock["record_id"]);r=tx(contract.functions.record(id,digest));logs=contract.events.Evid
ence().process_receipt(r);assert len(logs)==1;assert bytes(logs[0]["args"]["digest"])==digest;assert
contract.functions.recorded(id).call();reject(contract.functions.record(id,digest),a);(BASE/"indexed-
events.json").write_text(json.dumps([dict(id=id.hex(),digest=digest.hex(),block=r.blockNumber)],indent=2))
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Topic 4 Smart Contract Deployment

Mapped evidence: A6, A7, K5.

Activity 11 Publish a reproducible local deployment manifest

Goal

Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity workflow

Synthetic deployment-evidence record	CertificateNFT actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 251–255. Folder: labs/activity-11-deployment-evidence.

Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment.

Mapped learning evidence: A6, A7, K5. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace CertificateNFT against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Deployment manifest readback exact execution sequence

Deploy CertificateNFT locally, issue ID1 and check owner/code. Open deployment.json. Reconcile chainId/address/compiler/ABI/runtimeKeccak/sourceSha256 with the current source and get_code. Re-run default script: explain independent chain state, then use --rpc for persistent Anvil state.

Expected observable state: Nonempty runtime code; owner correct; manifest hashes represent current source and chain code.

Edit and verify task

Compare two runs where source changes; show source and runtime hashes differ, keeping old evidence explicitly separate. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build publish a reproducible local deployment manifest for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: Chain ID, address, ABI, bytecode hash and compiler lock uniquely describe a test deployment. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
contract CertificateNFT is ERC721 {
 address public immutable issuer=msg.sender;
 mapping(uint=>bytes32) public documentHash; mapping(uint=>bool) public revoked;
 constructor() ERC721("Course Certificate","CERT"){}
 function issue(address learner,uint id,bytes32 digest) external
{require(msg.sender==issuer,"issuer");documentHash[id]=digest;_safeMint(learner,id);}
 function revoke(uint id) external {require(msg.sender==issuer,"issuer");require(_ownerOf(id)!
=address(0),"missing");revoked[id]=true;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.issue(b,1,digest));assert len(w3.eth.get_code(contract.address))>0;assert
contract.functions.ownerOf(1).call()==b
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Activity 12 Connect a wallet and verify a certificate dApp

Goal

EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest.

Activity workflow

Synthetic wallet-dapp record	CertificateNFT actor/state control	Real EVM transaction / receipt	Python assertion and manifest

Test it

Test it: Require every positive receipt status1, exact state/units and every listed unauthorized/invalid EVM call rejected. Reconcile results.json and current deployment.json; browser/public testnet remain separately evidenced.

Slide reference: 256–260. Folder: labs/activity-12-wallet-dapp.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest.

Mapped learning evidence: A6, A7, K5. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm `python3 --version` and `node --version`. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with `python3 -m venv .venv`. Activate it using `source .venv/bin/activate` on macOS/Linux or `.venv\Scripts\activate` on Windows. Run `python -m pip install -r requirements.txt`.

5. Run `npm install` (then `npm ci` on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace CertificateNFT against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run `node compile.cjs`. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run `python run_activity.py`. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using `anvil --chain-id 31337 --host 127.0.0.1 --port 8545`. Keep the process running and use only its disposable default test accounts. Run `python run_activity.py --rpc http://127.0.0.1:8545`. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.

- If imports fail, activate the activity .venv and reinstall requirements; use `python -m pip` so pip and python refer to the same interpreter.

- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.

- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.

- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.

- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.

- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Wallet certificate dApp exact execution sequence

Launch persistent Anvil with chain31337. Run python run_activity.py --rpc http://127.0.0.1:8545. Serve this folder with python -m http.server8000. Open index.html, connect MetaMask using that local network. Load the same deployment.json, verify chain and query certificate ID1 ownership/digest/revoked. Switch wallet to Sepolia and verify the dApp blocks chain mismatch.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected observable state: Wallet readbacks match manifest; digest match true; revoked false; mismatch blocked before calls.

Edit and verify task

Change certificate ID to missing2 and show the revert/error message without falsely displaying valid status. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.

Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build connect a wallet and verify a certificate dapp for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: EIP-1193 provider checks chain ID then eth_call ownerOf/documentHash; explicit local-RPC deployment provides manifest. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Solidity source and review

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
contract CertificateNFT is ERC721 {
 address public immutable issuer=msg.sender;
 mapping(uint=>bytes32) public documentHash; mapping(uint=>bool) public revoked;
 constructor() ERC721("Course Certificate","CERT"){}
 function issue(address learner,uint id,bytes32 digest) external
{require(msg.sender==issuer,"issuer");documentHash[id]=digest;_safeMint(learner,id);}
 function revoke(uint id) external {require(msg.sender==issuer,"issuer");require(_ownerOf(id)!
=address(0),"missing");revoked[id]=true;}
}
```

Python acceptance assertions

```
from pathlib import Path
import csv, json, hashlib, argparse
from web3 import Web3, EthereumTesterProvider
from web3.exceptions import ContractLogicError
from eth_tester import EthereumTester, PyEVMBackend
from eth_tester.exceptions import TransactionFailed
BASE=Path(__file__).resolve().parent
args=argparse.ArgumentParser()
args.add_argument("--rpc",help="Optional isolated local JSON-RPC URL; default is real in-process Py-EVM")
opt=args.parse_args()
w3=Web3(Web3.HTTPProvider(opt.rpc)) if opt.rpc else Web3(EthereumTesterProvider(EthereumTester(PyEVMBackend())))
assert w3.is_connected()
accounts=w3.eth.accounts
assert len(accounts)>=3, "Supply isolated unlocked local test accounts"
a,b,c=accounts[:3]
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
w3.eth.default_account=a
art=json.loads((BASE/"contract.json").read_text())
receipts=[]
def tx(fn,sender=a,value=0):
 h=fn.transact({"from":sender,"value":value,"gas":3000000})
 r=w3.eth.wait_for_transaction_receipt(h);assert r.status==1, "EVM
reverted";receipts.append(dict(hash=h.hex(),gasUsed=r.gasUsed,block=r.blockNumber));return r
def reject(fn,sender=c,value=0):
 try: fn.call({"from":sender,"value":value})
 except (TransactionFailed, ContractLogicError): return
 except ValueError as exc:
  if "execution reverted" in str(exc).lower(): return
  raise
 raise AssertionError("Unauthorized/invalid action unexpectedly accepted")
def deploy(name=None,ctor=()):
 data=art if not name else json.loads((BASE/(name+".json")).read_text())
 fac=w3.eth.contract(abi=data["abi"],bytecode=data["bytecode"])
 r=tx(fac.constructor(*ctor));return w3.eth.contract(address=r.contractAddress,abi=data["abi"])
mock=json.loads((BASE/"mock-data.json").read_text())
digest=hashlib.sha256(json.dumps(mock,sort_keys=True,separators=(",",":")).encode()).digest()
contract=deploy();tx(contract.functions.issue(b,1,digest));assert contract.functions.ownerOf(1).call()==b;assert
bytes(contract.functions.documentHash(1).call())==digest
manifest={"chainId":w3.eth.chain_id,"address":contract.address,"contract":art["abi"],"compiler":art["compiler"],"evmVersion":art[
"evmVersion"],"sourceSha256":hashlib.sha256((BASE/"Contract.sol").read_bytes()).hexdigest(),"runtimeKeccak":Web3.kecca
k(w3.eth.get_code(contract.address)).hex(),"certificateId":1,"documentDigest":"0x"+digest.hex(),"learner":b,"rpc":opt.rpc,"back
end":"HTTP JSON-RPC" if opt.rpc else "Py-EVM"}
(BASE/"deployment.json").write_text(json.dumps(manifest,indent=2))
report={"activity":BASE.name,"backend":manifest["backend"],"chainId":w3.eth.chain_id,"contractAddress":contract.address,"as
sertions":"passed","receipts":receipts,"boundary":"Browser wallet and public testnet are separate optional checks"}
(BASE/"results.json").write_text(json.dumps(report,indent=2));print(json.dumps(report,indent=2))
```

Cross-check each assertion with its original business requirement. results.json reflects the supplied synthetic local run; preserve your own current evidence when executing the activity.

Detailed Ethereum Sepolia deployment and wallet evidence

This route supports the original PP public test-chain deployment task. It is a learner procedure and is not claimed as executed by the courseware build. Use disposable training wallets and test ETH only. Public network, wallet signing and Etherscan verification require separate current evidence.

1. Install MetaMask from its official website and create a new disposable training wallet. Record your recovery material privately; never paste it into prompts, source files or assessment evidence. Enable the Ethereum Sepolia test network in the wallet. Check chain ID 11155111.

2. Obtain test ETH from a Sepolia faucet linked by the current official Ethereum testnet documentation or your trainer. Verify the testnet wallet balance. Do not purchase mainnet ETH for this activity.

3. Open https://remix.ethereum.org. Create a workspace and upload Contract.sol from Activity 04 or 05. For an OpenZeppelin import, use the pinned version syntax supported by Remix, for example @openzeppelin/contracts@5.4.0/token/ERC20/ERC20.sol, and record the resolved dependency version. Retain the unmodified local source separately and record the import-path difference.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. In Solidity Compiler choose 0.8.30, enable optimizer with 200 runs and set EVM version Cancun. Compile the intended contract. Read each warning and error; retain a screenshot with selected compiler and contract name.

5. Open Deploy and Run Transactions. Choose the injected browser provider connected to MetaMask. Confirm the displayed account, network Sepolia and chain ID 11155111. Select WorkshopToken or CertificateNFT. Review any constructor fields; these two baseline contracts have no parameters.

6. Choose Deploy, inspect the MetaMask transaction request, network and fee estimate, then approve. Wait for the actual receipt. Capture the transaction hash, block number, receipt status and deployed address. A wallet approval or pending transaction is not deployment evidence.

7. Open https://sepolia.etherscan.io using the deployed address and transaction hash. Confirm successful creation and nonempty contract code. Record the chain, address, source/compiler lock, deployment timestamp and receipt. If explorer verification is requested, use the same source, optimizer, compiler and constructor settings; verify compilation metadata rather than merely publishing source text.

8. For WorkshopToken, call name, symbol, decimals, totalSupply and balanceOf(deployer). Import a custom token in MetaMask using the Sepolia contract address. Transfer 500 credits to a second disposable training address. Confirm integer units, recipient balance and transfer receipt; record the explorer URL.

9. For CertificateNFT, issue the synthetic record digest to a second training address with token ID1. Confirm ownerOf(1), documentHash(1) and revoked(1). Check an outsider issue attempt is rejected. Revoke ID1 and demonstrate the changed status without treating token ownership as valid certification.

10. Create a public-deployment manifest separately from local deployment.json. Include chainId=11155111, address, ABI, source hash, compiler, optimizer, dependency version, transaction hash, receipt and explorer URL. Do not label default local results as Sepolia evidence.

11. If wallet submission fails, inspect network, account, test ETH, gas estimate and the actual revert reason. If the address shows no code, confirm the transaction receipt and chain before retrying. Never change to a different public network without updating and documenting every manifest field.

12. Submit the required source, test log, manifest, wallet/token evidence and Etherscan creation/transfer evidence. Explicitly identify anything pending or not executed. Disconnect the training dApp and keep the recovery material private.

Original v17 content and technical correction register

The following text is copied from every original v17 slide. The current deck retains corresponding material in technical review slides and full source notes. It is a traceability appendix, not a instruction to execute retired tooling. Corrections: Ethereum mainnet now uses proof of stake; Rinkeby/Ropsten are retired; original Truffle/Ganache activities are historical and are replaced by pinned solc-js plus actual local Py-EVM/Anvil routes. On-chain digests prove consistency with input evidence, not the physical truth of a batch or an institution’s authority. Current ERC standards use the pinned OpenZeppelin implementation, not abbreviated legacy interfaces. Check current official docs for tool/network changes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 1

Develop Blockchain and Web3 App with Vibe Coding
Version: 17.0
Trainer: _____
Course Code: TGS-2021009338
Website:www.tertiarycourses.com.sg
Email: enquiry@tertiaryinfotech.com
Tel: +65 6100 0613

Original slide 2

It is mandatory for you to take both AM, PM and Assessment digital attendance for WSQ funded courses.

The trainer or administrator will show you the digital attendance QR code generated from SSG portal.

Please scan the QR code from your mobile phone camera and submit your attendance.
Digital Attendance (Mandatory)

Original slide 3

About the Trainer

Original slide 4

Let's Know Each Other...

Original slide 5

Ground Rules
Set your mobile phone to silent mode
Participate actively in the class. No question is stupid.
Mutual respect. Agree to disagree.
One conversation at one time.
Be punctual. Back from breaks on time.
Exit the class silently if you need to step out for phone call, toilet break etc.
75% attendance is required

Original slide 6

Lesson Plan
Day 1
Digital Attendance (AM)
Trainer and Learners Introduction

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Learning Outcomes
Course Outline
Topic 1: Blockchain, Cryptocurrency, Smart Contract and Token Economics
Lunch Break
Digital Attendance (PM)
Topic 2: Ethereum Blockchain and Solidity Programming
End of Day 1

Original slide 7

Lesson Plan
Day 2
Digital Attendance (AM)
Topic 3: Smart Contract Testing
Lunch Break
Digital Attendance (PM)
Topic 4: Smart Contract Deployment
Course Feedback and TRAQOM Survey
Digital Attendance (Assessment)
Final Assessment
End of Class

Original slide 8

TSC Title: Programming and Coding
TSC Code: FSE-DIT-4018-1.1

Skills Framework

Original slide 9

TSC Abilities:
A1: Gather and interpret business requirements of software through business use cases
A2: Review and propose overall coding best practices to guide individuals in executing coding

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A3: Review and detect errors and areas for improvement within written codes and communicate accordingly
A4: Test completed codes to identify gaps and possible improvement areas
A5: Manage programming and coding activities and monitor outputs
A6: Group coding outputs using a systematic approach
A7: Ensure programming and coding documentation is accurate and complete

TSC Knowledge:
K1: Business requirement analysis and mapping tools and/or techniques
K2: Team, departmental and organisational goals and strategic direction setting approaches
K3: Coding and code database organisation methodologies
K4: Information gathering and questioning techniques
K5: Trends and developments relating to programming and coding

Skills Framework TSC

Original slide 10

Learning Outcomes
LO1: Analyze business requirements for Blockchain, Cryptocurrency, Smart Contract and Token Economics.
LO2: Code smart contract for Ethereum blockchain using Solidity programming
LO3: Test smart contract and monitor its outputs
LO4: Deploy smart contract and ensure coding documentation is accurate

Original slide 11

Course Outline
Topic 1: Blockchain, Cryptocurrency, Smart Contract and Token Economics (A1, K1, K2)
Introduction of Blockchain and Cryptocurrency
Use Cases of Blockchain and Cryptocurrency
Introduction to Smart Contract and Non Fungible Token (NFT)
Use Cases of Smart Contract and NFT

Topic 2: Ethereum Blockchain and Solidity Programming
(A2, A3, K3)
Overview of Ethereum DApp Platform
Create Smart Contract using Solidity Programming
Solidity Coding Syntax
Best Practices on Solidity Coding

Topic 3: Smart Contract Testing(A4, A5, K4)
Test Smart Contract and Monitor Outputs
Manage Ether and Gas Fee

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Monitor Ethereum Blockchain Transactions

Original slide 12

Course Outline
Topic 4: Smart Contract Deployment (A6, A7, K5)
Deploy Smart Contract
Token Standards

Original slide 13

Final Assessment
Written Assessment (Short Answer Questions)
Practical Performance

Assessment format: Open Book
Open book assessment ONLY includes Slides, Learner Guide or any approved materials.
Appeal process

Original slide 14

Briefing for Assessment
Place phones & other materials under the table or on the floor
No photos or recording of assessment scripts
No discussion during assessment
Use black/blue pen for assessment [hard copies]
No usage of liquid paper or correction tape
Assessment scripts will be collected when the time is up

Original slide 15

Criteria for Funding
Minimum attendance rate of 75% based on SSG Digital Attendance record.
Complete the assessment and be assessed as ‘Competent’

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 16

LMS/TMS

https://lms-tms.tertiaryinfotech.com

Original slide 17

Topic 1
Blockchain, Cryptocurrency, Smart Contract and Token Economics

Original slide 18

What is Blockchain?

Blockchain is set of distributed data structure blocks, that is not editable or removable after published, linked by cryptographic validation in peer-to-peer network.
It is based on cryptographic ledgers which are treated in a decentralised and distributed manner.
It is are maintained and verified by the actors participating in the system, ensuring digital transparency and confidence in the records of information blocks.

Original slide 19

Traditional vs Blockchain Databases
While all records secured on a traditional database are centralized managed by database administrators, each participant on a blockchain has a secured copy of all records and there is no database administrators.

This new approach has greatly enhance the data immutability, transparency and security of the data.

Original slide 20

Risks and Challenges of Traditional Model
Single point of failure - all data is stored in a few centralized databases which are prompt to cyber hacking.

Fraud and security breaches - database administrators have full control over delegating access and granting various permissions to other users. With sufficient permissions, any one can freely alter information stored on the centralized server. If the security of the authority is compromised, the data can be altered, or even deleted.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lack of transparency and visibility - traditional database keep information up-to-date at a point in time. There is no real-time visibility into what changes are being made by multiple users at the same time.

Original slide 21

Trust is Important in Traditional Model

When you use traditional methods of payment, you need to trust in a third party to set your transaction (e.g. Visa, Mastercard, PayPal, banks, kickstarter, Lazada, Shoppe etc...)
What if the Trust is broken one day?

Original slide 22

History of Blockchain

Blockchain started its journey back in 1991.
2009 - Satoshi Nakamoto introduce the first generation of blockchain - Bitcoin blockchain
In 2013, a 19 years old Vitalik Buterin introduce the second generation of blockchain - Ethereum blockchain for smart contracts
Since then, many blockchains have been created

Original slide 23

Popular Blockchain Projects

and Many More!

Original slide 24

Web3

Web 1 is characterized by static information webpages
Web 2 is characterized by centralized internet platform and user interactivity .
Web 3 is characterized by decentralized apps that run on the blockchain.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 25

The Emergence of Web3

Original slide 26

Blockchains, Crypto and Tokens

Ethereum
PoW
Polygon
PoS
Bitcoin
PoW
Blockchains
Protocol

Ether
LINK
USTD
WBTC
OMG
SAND
MANA
MATIC

Blockchain Technology
Blockchains
Crypto and Tokens
Solana
PoH
SOL
ERC-20, ERC-721, ERC-1155

Original slide 27

Cryptocurrencies vs. Tokens

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

It’s important not to confuse the terms “cryptocurrencies” and “tokens,” as there are fundamental differences that distinguish them.
The biggest differentiation between the two is that cryptocurrencies have their own blockchains, whereas crypto tokens are built on an existing blockchain.

Original slide 28

Token Standards

There are several widely used token standards for creating crypto tokens, the majority of which have been built on top of Ethereum. Two most popular Ethereum tokens are:

Fungible tokens - ERC 20 standards.
Non-Fungible tokens - ERC 721 standards.

Original slide 29

Token Standards

Original slide 30

Bitcoin

Bitcoin (https://bitcoin.org/en/) was the first cryptocurrency created using Blockchain Technology and many people have perception that Bitcoin is Blockchain

Original slide 31

Bitcoin Wallet

A Bitcoin wallet is a type of digital wallet used to send and receive Bitcoins. This is analogous to a physical wallet.
However, instead of storing physical currency, the wallet stores the cryptographic information used to access Bitcoin addresses and send transactions.
Some Bitcoin wallets can also be used for other cryptocurrencies.

Original slide 32

The Birth of Blockchain in Bitcoin
In October 2008, Satoshi published the original whitepaper, describing his blueprint for a “A Peer-to-Peer Electronic Cash System.”
In January 2009 he released version 0.1 of the source code and launched the cryptocurrency by mining the genesis block.
Nakamoto was the first to solve the double-spending problem for a decentralized digital currency, creating a new asset, the likes of which the world had never seen before: Bitcoin.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 33

Who is Satoshi Nakamoto?

The name Satoshi Nakamoto is an alias used by the person or entity who created Bitcoin to hide their true identity.
The founder of Bitcoin continued to work on the software project for about another year and a half before disengaging from active development.

Original slide 34

What is a Block?

A block can be thought of like a link or node in a chain.
It consists of at least 3 main components
Data
Hash
Hash of previous block
The data contains all of the records of the transactions Eg Buyer, seller, and the amount.

Original slide 35

Hashing

Hashing means taking an input string of any length and giving out an output of a fixed length.
In the context of cryptocurrencies like Bitcoin, the transactions are taken as input and run through a hashing algorithm (Bitcoin uses SHA-256) which gives an output of a fixed length.
The hash can be thought of like the fingerprint of the block.

Original slide 36

Hash Function
A hash function is any function that can be used to map data of arbitrary size onto data of a fixed size

Original slide 37

SHA-256 Hashing Function
SHA (Secure Hash Algorithm) is the one used in bitcoin and most blockchains.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

SHA-256 algorithm generates an almost-unique, fixed size 256-bit (32-byte) hash.
This makes it suitable for password validation, challenge hash authentication, anti-tamper, digital signatures.

File
SHA 256
Hash value

Original slide 38

Activity: SHA256 Hashing
https://demoblockchain.org/hash
Type "Blockchain is a trustless distributed ledger"
As you type, change the length of th Hash is it the same.
Change Blockchain to lowercase "blockchain" and see the change in hash.

Original slide 39

Linking the Blocks

To link up the blocks, each block will store the hash of the previous block.

Original slide 40

Activity: Linking the Blocks
Goto https://demoblockchain.org/block
Add the following to the data to Block 1:
Buyer: Alice
Seller: Bob
Transaction: 10000
To add the block to the blockchain, you need to mine the block.

Original slide 41

Mining

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

To further mitigate any data tampering by hackers, blockchain requires validation of a new block in order to add the block to a blockchain, This is known as Proof of Work (PoW)
In Bitcoin, it takes about 10 mins for each PoW.

Original slide 42

Activity: Blockchain
Goto https://demoblockchain.org/blockchain
Add the data "Buyer: Alice, Seller: Bob, Transaction: 10000" to Block 1
Add the data "Buyer: Charlie, Seller: Eve, Transaction: 20000" to Block 2
Add the data "Buyer: Alice, Seller: Eve, Transaction: 30000" to Block 3
Mine the 3 blocks
Change Transaction from 20000 to 20001 in Block 2 and see what happen to the block 2 and 3.
Note that the prev hash is the same value as the has has in previous block

Original slide 43

Consensus Mechanism
Public blockchains that operate as decentralized, self-regulating systems work on a global scale without any single authority.
They involve contributions from hundreds of thousands of participants who work on verification and authentication of transactions occurring on the blockchain, and on the block mining activities.
In such a dynamically changing status of the blockchain, these publicly shared ledgers need an efficient, fair, real-time, functional, reliable, and secure mechanism to ensure that all the transactions occurring on the network are genuine and all participants agree on a consensus on the status of the ledger.
This all-important task is performed by the consensus mechanism, which is a set of rules that decides on the legitimacy of contributions made by the various participants (i.e., nodes or transactors) of the blockchain.

Original slide 44

Consensus Mechanisms

Original slide 45

Proof of Work (PoW)
Blockchains use Proof-of-Work, or PoW, consensus algorithms to elect a leader who will decide the contents of the next block.
This algorithm is used to confirm transactions and produce new blocks to the chain.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 46

Proof of Stake (PoS)

With proof-of-stake (POS), cryptocurrency owners validate block transactions based on the number of coins a validator stakes.
Proof-of-stake reduces the amount of computational work needed to verify blocks and transactions
Validators are then selected randomly to "mine," or validate the block. This system randomizes who gets to "mine" rather than using a competition-based mechanism like proof-of-work.

Original slide 47

Nonce and Difficulty Level
A nonce is an abbreviation for "number only used once," which, in the context of cryptocurrency mining, is a number added to a hashed—or encrypted—block in a blockchain that, when rehashed, meets the difficulty level restrictions.
The nonce is the number that blockchain miners are solving for. When the solution is found, the blockchain miners are offered cryptocurrency in exchange.

Original slide 48

Activity: Nonce
Goto https://demoblockchain.org/blockchain
Add the data "Buyer: Alice, Seller: Bob, Transaction: 10000" to Block 1
Manually change the nonce to obtain first 4 digit to be 0000 in the hash
Then press the Mine button to find the correct nonce.

Original slide 49

Peer to Peer Distributed Network
The PoW is carried out on a Peer to Peer (P2P) distributed network.
Whenever there is a new block, it will be send to every peer in the blockchain.
The miners will validate the block via PoW.
Once validated, the block will be added to the blockchain. The winning miner will be rewarded with bitcoins.
In Blockchain, a P2P network has replaced the role the central authority of a centralized database.

Original slide 50

Activity: Peer to Peer Network
Goto https://demoblockchain.org/distributed
The blockchains for all 3 peers are the same.
Assume we temper Block 3 for Peer 1 by adding some new data
Mine the Block 3, 4, 5 blockchain for Peer 1
Notice the hashes of Peer 1 are different from Peer 2 and Peer 3.
Therefore the community will know Peer 1 Block is being tempered.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Tampered!

Original slide 51

Immutable
One of the attributes that the blockchain is often associated with is its immutability.
If the data of a block is tampered, the hash will change, this will cause a mismatch in the hash.
So a hacker would need to change the hashes for all the subsequent blocks to avoid being detected!
This make the data immutable.

Original slide 52

Activity: Immutable
Goto https://demoblockchain.org/tokens
The blockchains for all 3 peers are the same.
Assume we temper Block 3 for Peer 1 by changing the transaction fee from $5 to $7..
Mine the Block 3, 4, 5 blockchain for Peer 1
Notice the hashes of Peer 1 are different from Peer 2 and Peer 3.
Therefore the community will know Peer 1 Block is being tempered

Tampered!

Original slide 53

Impact of Blockchain on Industry

The applications of blockchain technology are far beyond virtual currency and the financial industry.
This technology have disrupt many sectors such as below:

Original slide 54

Blockchain Applications

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 55

DeFi

Decentralized finance (DeFi) is an emerging financial technology based on secure distributed ledgers similar to those used by cryptocurrencies.
The system removes the control banks and institutions have on money, financial products, and financial services.

Original slide 56

DeFi Market Size

https://www.defipulse.com/

Original slide 57

DeFi Loan and Lending Platforms

https://loanscan.io/

Original slide 58

TokenSets

https://www.tokensets.com/

Original slide 59

What is Terra Stablecoins?

Original slide 60

The Future of DeFi

Decentralized Finance has already seen phenomenal growth over the last few years. However, there are over 1.7 billion unbanked people in the World excluded from the traditional financial system who could benefit from DeFi.
Imagine if you are living in a developing country with high inflation. DeFi gives you the opportunity to get out of your local currency and hold USD via stable coins like Dai. Even better, you can then take that Dai and lend it out at 5+% interest on DeFi platforms like Compound Finance. DeFi already offers people the tools to hedge out local currency risk and that’s a pretty big deal in countries like Venezuela.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Although DeFi loans currently need to be over-collateralized, it will only be a matter of time befer under-collateralized loans are available via a form of DeFi borrowing reputation system

Original slide 61

Problems With DeFi

It’s hard for the average person to access DeFi and that's due to friction between the crypto and traditional finance ecosystem. .
Another major issue facing DeFi are smart contract bugs. These could result in DeFi users losing access to a significant amount of funds and it’s even more worrying seeing that bugs are not unheard of in smart contracts.

Original slide 62

Crypto Exchanges

https://coinmarketcap.com/rankings/exchanges/

Original slide 63

Binance

https://www.binance.com

Original slide 64

Coinbase

https://www.coinbase.com/

Original slide 65

Gemini

https://www.gemini.com/share/gyx5ekvtx

Original slide 66

Original slide 67

source: https://www.channelnewsasia.com/business/singapore-based-cryptocurrency-exchange-cryptocom-hacked-more-400-users-affected-2450671

Original slide 68

source:" https://www.coindesk.com/markets/2019/03/26/singapore-based-crypto-exchange-dragonex-has-been-hacked/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 69

Topic 2
Ethereum Blockchain and Solidity Programming

Original slide 70

What is Ethereum ?

Ethereum (https://www.ethereum.org/) is a new generation of blockchain (first generation is the bitcoin).
It is an open-source, public, blockchain-based distributed computing platform featuring smart contract functionality.
It is a programmable Blockchain platform for Decentralized Applications (DApps) that run on Ethereum Virtual Machine (EVM)
Developers can create smart contract DApps on EVM using Solidity programming language

Original slide 71

History of Ethereum

Ethereum was proposed in late 2013 by Vitalik Buterin
Development was funded by an online crowdsale that took place between July and August 2014.
Ethereum went live on 30 July 2015

Original slide 72

How Ethereum Work?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Contracts written in a smart contract-specific programming languages eg Solidity are compiled into ‘bytecode’ that can run on ‘ethereum virtual machine’ (EVM)

Original slide 73

Smart Contract
Smart Contracts are sets of Business Rules in the system.
All smart contracts are verified and signed by participants.
When certain conditions are met, auto execute it with together the transaction
Ethereum is the first blockchain platform that implement smart contracts in Dapp

Original slide 74

Overview of Smart Contract

Original slide 75

A dapp has its backend code running on a decentralized peer-to-peer network.
You can explore the DApps created on Ethereum here
https://www.stateofthedapps.com/

What is a DApp

Original slide 76

Ether (ETH) Unit
Ether can be broken down to smaller units:

Original slide 77

Ether Price

https://finance.yahoo.com/chart/ETH-USD

Original slide 78

Etherscan is website that monitors all the Ethereum transactions and tokens.
https://etherscan.io/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Etherscan

Original slide 79

Goto https://etherscan.io/
Look at the Ethereum transaction
Goto one of the Ethereum block and look at the data inside the block

Activity: Ethereum Transactions

Original slide 80

Crypto Wallet

A crypto wallet, such as MetaMask, stores your ETH and processes transactions on the Ethereum blockchain.
A unique wallet address will be generated and you will use this address to complete transactions..

Original slide 81

Ethereum Wallet - MetaMask
MetaMask (https://metamask.io/) is a browser extension that provide a way for way for normal web browser to access the blockchain and propose transactions.
It injects web3 API into every website you visit.
It allows you to run Ethereum dApps right in your browser without running a full Ethereum node

Original slide 82

MetaMask Wallet
https://metamask.io/

Original slide 83

Activity: Install Metamask Wallet
Goto https://metamask.io/download/
Install Metamask Wallet

Original slide 84

Mainnet vs Tesnet

Mainnet is the term used to describe when a blockchain protocol is fully developed and deployed, meaning that cryptocurrency transactions are being broadcasted, verified, and recorded on a distributed ledger technology blockchain.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A testnet is used by programmers and developers to test and troubleshoot all the aspects and features of a blockchain network before they are sure the system is secure and ready for the mainnet launch.

Original slide 85

Ethereum Testnets

Görli
Kovan
Rinkeby
Ropsten

ETH on testnets has no real value; therefore, there are no markets for testnet ETH. Since you need ETH to actually interact with Ethereum, most people get testnet ETH from faucets.

Original slide 86

Activity: Get Testnet Ethers
You can get Rinkeby testnets ethers from the faucets below
https://faucet.rinkeby.io/
Send the testnet ethers to each other in the class

Original slide 87

Ethereum Gas Fee

Original slide 88

Beware of Ethereum Gas Price

Original slide 89

Create a 2nd test account on Rinkeby Test Network
Send 5 ether from the 1st test account to 2nd test account
Specify the gas price used

Activity: Ether Transaction and Gas

Original slide 90

Transaction Details on Etherscan

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 91

Activity: Etherscan

View all your test transactions on Rinkeby Etherscan https://rinkeby.etherscan.io/

Original slide 92

Solidity
Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs which govern the behaviour of accounts within the Ethereum state.
Solidity was influenced by C++, Python and JavaScript and is designed to target the Ethereum Virtual Machine (EVM).
Solidity is statically typed, supports inheritance, libraries and complex user-defined types among other features.
With Solidity you can create contracts for uses such as voting, crowdfunding, blind auctions, and multi-signature wallets.

Original slide 93

Remix IDE for Solidity
Remix is a suite of tools to interact with the Ethereum blockchain in order to debug transactions, stored in this Git repository
Remix IDE is a browser-based IDE built by the Ethereum development team.
http://remix.ethereum.org

Original slide 94

Layout of a Solidity Source File
Source files can contain an arbitrary number of contract definitions, import directives, pragma directives and struct, enum, function, error and constant variable definitions.
A typical layout of solidity source file includes:
SPDX License Identifier
Pragmas
Contract
Comments - Single-line comments (//) and multi-line comments (/*...*/) are possible

Original slide 95

Sample Smart Contract

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Storage {
    uint256 number;
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    function store(uint256 num) public {
        number = num;
    }

    function retrieve() public view returns (uint256){
        return number;
    }
// This is a comment
}
```

Original slide 96

SPDX License Identifier
Since making source code available always touches on legal problems with regards to copyright, the Solidity compiler encourages the use of machine-readable SPDX license identifiers.
Every source file should start with a comment indicating its license

For example

```
// SPDX-License-Identifier: MIT
// SPDX-License-Identifier: GPL-3.0
```

Original slide 97

Pragmas
The pragma keyword is used to enable certain compiler features or checks.
A pragma directive is always local to a source file, so you have to add the pragma to all your files if you want to enable it in your whole project.
If you import another file, the pragma from that file does not automatically apply to the importing file.

For Example:

```
pragma solidity ^0.5.2;
pragma solidity >=0.7.0 <0.9.0;
```

Original slide 98

Contract
Contracts in Solidity are similar to classes in object-oriented languages.
Contracts in Solidity is a collection of code (its functions) and data (its state)
Each contract can contain declarations of State Variables, Functions, Function Modifiers, Events, Struct Types and Enum Types.
Contracts can inherit from other contracts.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 99

Structure of a Contract
Contract in Solidity is similar to a Class in C++. A Contract have following properties.
Constructor − A special function declared with constructor keyword which will be executed once per contract and is invoked when a contract is created.
State Variables − Variables per Contract to store the state of the contract.
Functions − Functions per Contract which can modify the state variables to alter the state of a contract.

Original slide 100

Solidity Contract Structure

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.6;

<import library>

contract ContractName {
<state variable declaration>
<mapping>

<constructor>
<modifiers>

<functions>
}
```

Original slide 101

Hello World Contract Demo

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

contract Hello {

   uint256 public myNum;

   function store(uint256 _myNum) public {
       myNum = _myNum;
   }


}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 102

Visibility Quantifiers
External − External functions are meant to be called by other contracts. They cannot be used for internal call. To call external function within contract this.function_name() call is required. State variables cannot be marked as external.
Public − Public functions/ Variables can be used both externally and internally. For public state variable, Solidity automatically creates a getter function.
Internal − Internal functions/ Variables can only be used internally or by derived contracts.
Private − Private functions/ Variables can only be used internally and not even by derived contracts.

Original slide 103

Compile Smart Contract
Choose the Solidity compiler version
Compile the smart contract.
Compiler will output the bytecode and ABI

You can copy the Bytecode and ABI after you compile here

Original slide 104

The Contract Application Binary Interface (ABI) is the standard way to interact with contracts in the Ethereum ecosystem, both from outside the blockchain and for contract-to-contract interaction.
Data is encoded according to its type, as described in this specification.
The encoding is not self describing and thus requires a schema in order to decode

Contract ABI

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 105

Deploy Smart Contract
Choose the Javascript VM
Click on Deploy button

Original slide 106

Function Syntax
function functionName(arguments…) <visibility> <state mutability> returns <return types> {....}
Function return type
State Mutability
Function Visibility
function functionName(arguments…)
public/private/… pure/view/payable
returns <return types> {
        …..code your function logic here
}

Original slide 107

Store and Retrieve Number

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Hello {
   uint256 public myNum;

   function store(uint256 _myNum) public {
       myNum = _myNum;
   }

   function retrieve() public view returns(uint256) {
      return myNum;
   }

}
```

Original slide 108

Store and Retrieve String

```
// SPDX-License-Identifier: GPL-3.0
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
pragma solidity >=0.7.0 <0.9.0;

contract Hello {
   string public myString;

   function store(string memory _myString) public {
       myString = _myString;
   }

   function retrieve() public view returns(string memory) {
      return myString;
   }

}
```

Original slide 109

Constructor
Constructor is a special function declared using constructor keyword.
When a contract is created, its constructor (a function declared with the constructor keyword) is executed once.
Constructor is generally used to set the default values for state variables
A constructor is optional.
Only one constructor is allowed for a contract

Original slide 110

Contractor Demo

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Hello {
   uint256 number;

   constructor() {
      number = 10;
   }
   function store(uint256 num) public {
      number = num;
   }
   function retrieve() public view returns (uint256){
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
        return number;
    }
}
```

Original slide 111

Activity: Constructor
Create a constructor to assign a string value

Hint:

```
string myString;

constructor() {
        myString = "Hello";
    }
```

Original slide 112

Deploy Smart Contract on TestNet
To deploy your smart contract on Rinkeby, change the environment from Javascript VM to Injected Web3.
Each transaction will cost a gas fee.
View the transactions on Rinkeby Etherscan

Original slide 113

Activity: Deploy Contract to Testnet
Deploy the contract in previous constructor activity to Rinkeby testnet

Original slide 114

Simple Data Types
String
Integers (int or uint)
Bool
Address
Bytes

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Data Types
Complex Data Types
Array
Struct
Enum
Mapping

Original slide 115

Constant

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

contract MyContract {
    string public constant myString= "Hello";
}
```

Original slide 116

Simple Data Types Demo

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

contract MyContract {
    string public myString = "myString";
    bool public myBool = true;
    int public myInt = 1;
    uint public myUint = 1;
    uint8 public mUint8 = 8;
    uint256 public myUint265 = 9999;

}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 117

Mapping
Mapping is a reference type as arrays and structs. Following is the syntax to declare a mapping type.\

Syntax:
mapping(_KeyType => _ValueType)

Original slide 118

Mapping Demo

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.6;

contract Hello {

 mapping(string => uint256) public person;

 function addPerson(string memory _name, uint256 _age) public {
        person[_name] = _age;
    }
}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 119

Activity: Mapping
Create a mapping to people count to Person struct and to keep track of the people

Hint:

```
struct Person {
        uint _id;
        string _firstname;
        string _lastname;
    }

uint256 public peopleCount = 0;
mapping(uint => Person) public people;
```

Original slide 120

In Solidity there are special variables and functions which always exist globally and are mainly used to provide information about the blockchain.
They can also be used to assist with error handling, employ mathematical and cryptographic functions, and present information about contract addresses and the contracts themselves.
Common global variables are:
msg
gas
tx
block

Global Variables

Original slide 121

blockhash (uint blockNumber) returns (bytes32) - Hash of the given blocks.
block.coinbase (address payable) = Current block miner's address.
block.difficulty (uint) - current block difficulty.
block.gaslimit (uint) - Current block gaslimit.
block.number (uint) - Current block number.
block.timestamp - Current block timestamp as seconds since unix epoch.
gasleft() returns (uint256) - Remaining gas.
msg.data (bytes calldata) - Complete calldata.
msg.sender (address payable) - Sender of the message (current call).
msg.sig (bytes4) - First four bytes of the calldata (i.e. function identifier)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

msg.value (uint) - Number of wei sent with the message.
now (uint) - Current block timestamp (alias for block.timestamp).
tx.gasprice (uint) - Gas price of the transaction.
tx.origin (address payable) - Sender of the transaction (full call chain)

Global Properties

Original slide 122

Global Variable and Payable Demo

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.6.6 <0.9.0;


contract FundMe {

    mapping(address => uint256) public addrToAmt;

    function fund() public payable {
        addrToAmt[msg.sender] += msg.value;
    }

}
```

Original slide 123

Function Modifier
Function modifier is mainly used to check a condition (restriction) prior to executing the function.
You can think of function modifier as an extension of a function.
Modifier code will run first when assigned by a function.
_; means to return the flow of execution to the original function code.

Original slide 124

Function Modifier Demo

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.6.6 <0.9.0;
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
contract FundMe {
    mapping(address => uint256) public addrToAmt;
    address public owner;
    constructor() {
        owner = msg.sender;
    }
    modifier onlyOwner {
        require(msg.sender == owner);
        _;
    }
    function fund() public payable onlyOwner {
        addrToAmt[msg.sender] += msg.value;
    }
}
```

Original slide 125

Activity: Function Modifier
Add a function modifier to the previous activity on mapping that only the owner can add person to the lis.

Hint:

```
    mapping(uint => Person) public people;
    uint256 public peopleCount;
    address owner;

    modifier onlyOwner {
        require(msg.sender == owner);
        _;
    }

    constructor() {
        owner = msg.sender;
    }
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 126

The transfer() function of the ERC20 standard is used to transfer the tokens from the owner of the token to some other address.
As per the standard, the transfer() function must emit the Transfer event on a successful transferal of tokens.
The transaction should revert if msg.sender does not have enough tokens to spend.
A transfer of a 0 (zero) must also be treated as a valid transfer, and must log the Transfer event.
The address that's calling the function must have sufficient tokens to transfer to another address.
Transfer Function

Original slide 127

Transfer Function Demo

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.6.6 <0.9.0;

contract Token {
    mapping(address => uint256) public balances;
    address payable funder;
    address owner;

    constructor(address payable _funder)  {
       funder = _funder;
       owner = msg.sender;
    }
    modifier onlyOwner {
        require(msg.sender == owner);
        _;
    }
    function myToken() public payable onlyOwner {
      balances[msg.sender] += 1;
      funder.transfer(msg.value);
    }
}
```

Original slide 128

Activity: Transfer Function
Add a SellToken function when owner sell the token

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Hint:

```
function sellToken() public payable {
    balances[msg.sender] -= 1;
    payable(owner).transfer(msg.value);
  }
```

Original slide 129

Inheritance is a way to extend functionality of a contract. Solidity supports both single as well as multiple inheritance. Following are the key highlights.
A derived contract can access all non-private members including internal methods and state variables. But using this is not allowed.
Inheritance Syntax:

```
contract MyToken is ERC20Token
```

Inheritance

Original slide 130

The ERC-20 (Ethereum Request for Comments 20), proposed by Fabian Vogelsteller in November 2015, is a Token Standard that implements an API for tokens within Smart Contracts.
ERC-20 introduces a standard for Fungible Tokens, in other words, they have a property that makes each Token be exactly the same (in type and value) of another Token. For example, an ERC-20 Token acts just like the ETH, meaning that 1 Token is and will always be equal to all the other Tokens.
It provides functionalities like to transfer tokens from one account to another, to get the current token balance of an account and also the total supply of the token available on the network. Besides these it also has some other functionalities like to approve that an amount of token from an account can be spent by a third party account.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

ERC20 Token

Original slide 131

How ERC20 Token Works

This diagram illustrate the steps of how the process works.
First an ERC20 token is published to the ethereum network
The token is assigned a contract address and has an ABI
An address that holds the token approves a contract to spend on its behalf
The contract then sends the token to a receiving address

Original slide 132

https://eips.ethereum.org/EIPS/eip-20

ERC20 Token

Original slide 133

totalSupply() - Returns current total number of tokens
balanceOf() - Returns current balance of specified account
allowance() - Returns remaining number of tokens allowed to be transferred from one specific account to another specific account
transfer() - Transfers tokens from the owner to specified target account
transferFrom() - Transfers tokens from one specific account to another specific account
approve() - Maximum allowed tokens that can be transferred from one specific account to another specific account

Typical ERC20 Key Functions

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 134

totSupply - Total number of tokens in circulation
name - Descriptive token name
decimals - Number of decimals to use when displaying token amounts
symbol - Short identifier
balances - Current balance of each participating account, mapped to the account's address
allowed - Number of tokens authorized to transfer between accounts, mapped to sender's address

Typical ERC20 Data Items

Original slide 135

OpenZeppelin Contracts (https://docs.openzeppelin.com/) helps you minimize risk by using battle-tested libraries of smart contracts for Ethereum and other blockchains. It includes the most used implementations of ERC standards.

OpenZeppelin

Original slide 136

https://github.com/OpenZeppelin/openzeppelin-contracts/tree/master/contracts/token

OpenZeppelin Token Codes

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 137

Goto OpenZeppelin Contract Wizard page
https://docs.openzeppelin.com/contracts/4.x/wizard
Enter your Token Setting

Create ERC20 Token Smart Contract

Original slide 138

Copy the generated smart contract to a new file in Remix
Replace the import to
https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20/ERC20.sol
Compile the smart contract in Javascript
Test the transfer of tokens from one account to another account

Compile and Test Your ERC20 Token

Original slide 139

Deploy the ERC20 token smart contract on testnet via the Metamask

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Deploy ERC20 Token on Testnet

Original slide 140

Check the contract deployment on testnet etherscan
Copy the contract address

Review Token Contract on Etherscan

Original slide 141

Import the tokens to the metamask by entering the contract address.
Import ERC20 Token to Metamask

Original slide 142

Click on the tokens to send to other wallet address.

Send ERC20 Token to Others

Original slide 143

Create a ERC20 Token below
https://vittominacori.github.io/erc20-generator/create-token/
Enter your own Token name, Token symbol
Choose SimpleERC20 on Rinkeby /Ropsten testnet

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Create ERC20 Token

Original slide 144

Add the tokens to your Metamask
View your token transaction on testnet etherscan
Transfer some of you tokens to other

Transact ERC20 Token

Original slide 145

Activity: Create ERC20 Token
Code a ERC-20 token smart contract on Remix
Compile and deploy on Javascript VM
Test out by transfer your own token from one account to another account
Deploy the tokens on Ethereum testnet eg Rinkeby
Check the token contract on Etherscan

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Import your tokens to your metamask wallet
Transfer some of your tokens to others

Original slide 146

Topic 3
Smart Contract Testing

Original slide 147

The Ethereum protocol defines a way for people to interact with smart contracts and each other over a network.
In order to have up-to-date information about the status of contracts, balances, and new transactions, the protocol requires a connection to nodes on the network. These nodes are constantly sharing new data with each other.
Web3 is a framework for connecting to these nodes. It does not run its own node internally.

Ethereum Node and Web3

Original slide 148

Web3.js is a collection of libraries that allow you to interact with a local or remote ethereum node using HTTP, IPC or WebSocket.

Web3.js

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 149

Test networks usually have something called a “faucet” to help get test ether to people who want to use it. The faucet simply sends you test ether when you visit a web page, or ping a chat bot, etc.
Each test network has its own version of test ether, so each one must maintain its own faucet..
Here are some links to testnet ether instructions (in no particular order):
Kovan
Rinkeby
Ropsten

Test Networks

Original slide 150

The provider is how web3 talks to the blockchain. Providers take JSON-RPC requests and return the response. This is normally done by submitting the request to an HTTP or IPC socket based server.
The most common ways to connect to your node are:
IPC (uses local filesystem: fastest and most secure)
Websockets (works remotely, faster than HTTP)
HTTP (more nodes support it)

Providers

Original slide 151

The Ethereum protocol defines a way for people to interact with smart contracts and each other over a network. In order to have up-to-date information about the status of contracts, balances, and new transactions, the protocol requires a connection to nodes on the network. These nodes are constantly sharing new data with each other.
Web3.py is a python library for connecting to these nodes. It does not run its own node internally.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Web3.py

Original slide 152

Install Python from
https://www.python.org/downloads/
Open the terminal and install Web3 Python package as follows:

```
pip install web3
```

Mac
pip3 install web3

Install Web3 Python Package

Original slide 153

```
from web3 import Web3
w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:8545'))
```

The quickest way to interact with the Ethereum blockchain is to use a remote node provider, like Infura. You can connect to a remote node by specifying the endpoint, just like the previous local node example:

```
from web3 import Web3
w3 = Web3(Web3.HTTPProvider('https://mainnet.infura.io/v3/<infura-project-id>'))
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Using Web3.py

Original slide 154

Infura allows you to assess the Ethereum blockchain.
Sign up an account here https://infura.io/
Sign Up Infura

Original slide 155

Create a Infura project
Copy the endpoints key on mainnet
Get Endpoint

endpoint

Original slide 156

```
from web3 import Web3

# Fill in your infura API key here
infura_url = "https://mainnet.infura.io/v3/YOUR_INFURA_API_KEY_GOES_HERE"

web3 = Web3(Web3.HTTPProvider(infura_url))
print(web3.isConnected())
print(web3.eth.blockNumber)
```

Test Infura EP with Web3 Python

Original slide 157

```
web3.eth.blockNumber
web3.eth.gasPrice
web3.eth.default_account
web3.eth.get_balance('<address>')
web3.eth.get_transaction('<address>")
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Getting Blockchain Info

Original slide 158

Goto etherscan and get a block no
Use Web3.py to request for the block info

Activity: Web3.py

Original slide 159

You can use Web3 to check our Ethereum wallet balance as follows:

```
balance = web3.eth.getBalance("YOUR_ACCOUNT_GOES_HERE")
print(web3.fromWei(balance, "ether"))
```

Check Your Account Balance

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 160

https://etherscan.io/token/0xd26114cd6EE289AccF82350c8d8487fedB8A0C07

OMG Token

Original slide 161

```
import json
from web3 import Web3

# Fill in your infura API key here
infura_url = "https://mainnet.infura.io/v3/95408920a948489c99c5bcb436235bd2"
web3 = Web3(Web3.HTTPProvider(infura_url))

# OMG Address
abi = json.loads('[{"...."}]')
# OMG ABI
address = '0xd26114cd6EE289AccF82350c8d8487fedB8A0C07'

contract = web3.eth.contract(address=address, abi=abi)

totalSupply = contract.functions.totalSupply().call()
print(web3.fromWei(totalSupply, 'ether'))
print(contract.functions.name().call())
print(contract.functions.symbol().call())
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Get Info on OMG Token with Web3

Original slide 162

Use Web3.py to request the name and symbol of Bancor Token
https://etherscan.io/address/0x1f573d6fb3f13d689ff844b4ce37794d79a7ff1c#code

Activity: Bancor Token

Original slide 163

Ganache is a personal Ethereum blockchain which you can use to run tests, execute commands, and inspect state while controlling how the chain operates.
Download and install Ganache https://www.trufflesuite.com/ganache

Ganache - Personal Ethereum

Original slide 164

Setup Ganache
account
private key
Ganache network

Original slide 165

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
contract Hello {
    string public myString;

    function store(string memory _myString) public {
        myString = _myString;
    }

    function retrieve() public view returns(string memory) {
        return myString;
    }

}
```

Transact Contract on Ganache
Choose Web3 Provider
Set to Ganache network

Original slide 166

Test the following smart contract on Ganache

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Storage {
    uint256 myNum;

    constructor() {
        myNum = 5;
    }

    function store(uint256 num) public {
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
        myNum = num;
    }

    function retrieve() public view returns (uint256){
        return myNum;
    }
}
```

Activity: Transact on Ganache

Original slide 167

Ganache Blocks and Transactions

Original slide 168

```
# Step1: Create the transaction
tx = {
    'nonce': nonce,
    'to': account_2,
    'value': web3.toWei(5, 'ether'),
    'gas': 2000000,
    'gasPrice': web3.toWei('50', 'gwei'),
}

# Step2: Sign the transaction
signed_tx = web3.eth.account.signTransaction(tx, private_key)

# Step 3: Send the transaction
tx_hash = web3.eth.sendRawTransaction(signed_tx.rawTransaction)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Blockchain Transaction

Original slide 169

```
from web3 import Web3

ganache_url = "http://127.0.0.1:7545"
web3 = Web3(Web3.HTTPProvider(ganache_url))

account_1 = '' # Fill me in
account_2 = '' # Fill me in
private_key = '' # Fill me in

nonce = web3.eth.getTransactionCount(account_1)

tx = {
    'nonce': nonce,
    'to': account_2,
    'value': web3.toWei(1, 'ether'),
    'gas': 2000000,
    'gasPrice': web3.toWei('50', 'gwei'),
}
signed_tx = web3.eth.account.signTransaction(tx, private_key)
tx_hash = web3.eth.sendRawTransaction(signed_tx.rawTransaction)
print(web3.toHex(tx_hash))
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Transaction between 2 Accounts
Get the account and private key from Ganache

Original slide 170

Transactions on Ganache

Original slide 171

Transfer 5 ethers from account 2 to 3 in Ganache

Activity: Transactions on Ganache

Original slide 172

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Hello {
   string public myString;

   function store(string memory _myString) public {
       myString = _myString;
   }

   function retrieve() public view returns(string memory) {
      return myString;
   }

}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Deploy Test Contract on Remix

Original slide 173

```
import json
from web3 import Web3

# Set up web3 connection with Ganache
ganache_url = "http://127.0.0.1:7545"
web3 = Web3(Web3.HTTPProvider(ganache_url))

# Set a default account to sign transaction
web3.eth.defaultAccount = web3.eth.accounts[0]

# Contract ABI
abi = json.loads('XXXX')

# Contract address
address = web3.toChecksumAddress('XXXX')
```

Test Contract with Web3

Original slide 174

```
# Initialize contract
contract = web3.eth.contract(address=address, abi=abi)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
# Store value
tx_hash = contract.functions.store('Good Day').transact()

# Wait for transaction to be mined
web3.eth.waitForTransactionReceipt(tx_hash)

# Retrieve value
print(f'Retrive the value: {contract.functions.retrieve().call()}')
```

Test Contract with Web3

Original slide 175

Where to Get ABI and ByteCode

Original slide 176

Where to Get Contract Address
Click on the icon here

Original slide 177

Check Blocks on Ganache

Original slide 178

Deploy the following smart contract on Ganache and test with Web3

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract Storage {
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    uint256 myNum;

    constructor() {
        myNum = 5;
    }

    function store(uint256 num) public {
        myNum = num;
    }

    function retrieve() public view returns (uint256){
        return myNum;
    }
}
```

Activity: Test Transaction with Web3

Original slide 179

Topic 4
Smart Contract Deployment

Original slide 180

Develop DApp

To develop and deploy an DApp requires the following components:
smart contract (solidity)
deploy smart contract on Blockchain
Web3 to interact with Blockchain
Metamask to interface between Web3 and Browser
Node.js or React to build the frontend and backend server.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 181

DApp Development Frameworks

Truffle (Javascript)
Hardhat (Typescript)
Brownies (Python)

Original slide 182

Install Truffle

Install node.js (https://nodejs.org/en/)
Install Truffle by typing the following CML:

```
npm install -g truffle
```

Original slide 183

Start a Simple Truffle Project (Optional)

Creating a project as follows in CML
mkdir hello
cd hello
truffle init
Open the project from VS Code
Create a new file hello.sol under contract folder and insert your hello.sol script
Change your compiler version at truffle-config.js
Compile the smart contract as follows in CML:
truffle compile

Original slide 184

Deploy a Simple Truffle Project

Change the migration script to:

```
const Hello= artifacts.require("hello");
module.exports = function (deployer) {
  deployer.deploy(Hello);
};
```

Change the network setting to Ganache:

```
 development: {
      host: "127.0.0.1",
      port: 7545,
      network_id: "*",
     },
```

Deploy the smart contract as follows in CML:
truffle deploy -reset

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 185

Test a Simple Truffle Project

Test a truffle project as follows in CML
truffle console
Type the following to the console environment:

Hello.deployed().then(function(instance) {return instance.store(255) });
Hello.deployed().then(function(instance) {return instance.retrieve() });

Original slide 186

Activity: Pet Shop Tutorial (Optional)

This tutorial (https://www.trufflesuite.com/tutorial) will take you through the process of building your first dapp---an adoption tracking system for a pet shop!
This tutorial is meant for those with a basic knowledge of Ethereum and smart contracts, who have some knowledge of HTML and JavaScript,
In this tutorial we will be covering:
Setting up the development environment
Creating a Truffle project using a Truffle Box
Writing the smart contract
Compiling and migrating the smart contract
Testing the smart contract
Creating a user interface to interact with the smart contract
Interacting with the dapp in a browser

Original slide 187

Get Started on Pet Shop Tutorial

Creating a project as follows in CML
mkdir pet-shop
cd pet-shop
truffle unbox pet-shop

Open the project from VS Code

Original slide 188

Adoption.sol

Create a new file named Adoption.sol in the contracts/ directory and add the following content:

```
pragma solidity ^0.5.0;
contract Adoption {
   address[16] public adopters;
   // Adopting a pet
   function adopt(uint petId) public returns (uint) {
      require(petId >= 0 && petId <= 15);
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
        adopters[petId] = msg.sender;
        return petId;
        }
    // Retrieving the adopters
    function getAdopters() public view returns (address[16] memory) {
        return adopters;
        }
  }
```

Update the compiler version in truffle-config.js
truffle compile

Original slide 189

Migration Script

Replace the migration script to:

```
var Adoption = artifacts.require("Adoption");

module.exports = function(deployer) {
  deployer.deploy(Adoption);
};
```

Update the network info
truffle deploy -reset

Original slide 190

Creating a User Interface to Interact with
the Smart Contract

Now that we've created the smart contract, deployed it to our local test blockchain and confirmed we can interact with it via the console, it's time to create a UI so that Pete has something to use for his pet shop!
Included with the pet-shop Truffle Box was code for the app's front-end. That code exists within the src/ directory.

Original slide 191

Instantiating Web3

Open /src/js/app.js in a text editor.
Remove the multi-line comment from within initWeb3 and replace it with the following:

Original slide 192

Instantiating Web3

Still in /src/js/app.js, remove the multi-line comment from within initContract and replace it with the following:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Original slide 193

Instantiating Web3

Still in /src/js/app.js, remove the multi-line comment from markAdopted and replace it with the following:

Original slide 194

Instantiating Web3

Still in /src/js/app.js, remove the multi-line comment from handleAdopt and replace it with the following:

Original slide 195

Create New Network in Metamask

In the box titled "New Network" enter http://127.0.0.1:7545, in the box titled "Chain ID" enter 1337 (Default Chain ID for Ganache) and click Save

Original slide 196

Import Account from Ganache

Click the top-right X to close out of Settings and return to the Accounts page.
Import Account and enter your private key from Ganache
Each account created by Ganache is given 100 ether. You'll notice it's slightly less on the first account because some gas was used when the contract itself was deployed and when the tests were run.

Original slide 197

Run the DApp

Enter the following command in CML
npm run dev
A MetaMask pop-up should appear requesting your approval to allow Pete's Pet Shop to connect to your MetaMask wallet. Without explicit approval, you will be unable to interact with the dapp. Click Connect.

Original slide 198

Test the DApp

To use the dapp, click the Adopt button on the pet of your choice.
You'll be automatically prompted to approve the transaction by MetaMask. Click Submit to approve the transaction.

Original slide 199

Test out Metacoin on Truffle as follows

mkdir MetaCoin
cd MetaCoin
truffle unbox metacoin

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

truffle compile
Open truffle-config.js in a text editor. Replace the content with the following:

```
module.exports = {
  networks: {
    development: {
      host: "127.0.0.1",
      port: 7545,
      network_id: "*"
    }
  }
};
```

Activity: MetaCoin on Truffle

Original slide 200

On the terminal, migrate the contract to the blockchain created by Ganache:
truffle migrate
To interact with the contract, you can use the Truffle console.
truffle console
Begin by establishing both the deployed MetaCoin contract instance and the accounts created by either Truffle's built-in blockchain or Ganache:
let instance = await MetaCoin.deployed()
let accounts = await web3.eth.getAccounts()
Check the metacoin balance of the account that deployed the contract:
let balance = await instance.getBalance(accounts[0])
balance.toNumber()
See how much ether that balance is worth (and note that the contract defines a metacoin to be worth 2 ether):
let ether = await instance.getBalanceInEth(accounts[0])
ether.toNumber()

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Transfer some metacoin from one account to another:
instance.sendCoin(accounts[1], 500)

Activity: MetaCoin on Truffle

Original slide 201

Check the balance of the account that received the metacoin:
let received = await instance.getBalance(accounts[1])
received.toNumber()
Check the balance of the account that sent the metacoin:
let newBalance = await instance.getBalance(accounts[0])
newBalance.toNumber()

Activity: MetaCoin on Truffle

Original slide 202

Summary
Q&A

Original slide 203

Cert & TRAQOM Survey (Mandatory)
https://lms-tms.tertiaryinfotech.com/

Original slide 204

It is mandatory for you to take both AM, PM and Assessment digital attendance for WSQ funded courses.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The trainer or administrator will show you the digital attendance QR code generated from SSG portal.

Please scan the QR code from your mobile phone camera and submit your attendance.
Digital Attendance (Mandatory)

Original slide 205

Final
Assessment

Original slide 206

Recommended Courses
WSQ - Business Innovation with Blockchain Technology
WSQ - Mastering HTML5 and CSS3 Fundamentals for Professional Web Design
WSQ - Hands-On Web App Development with Javascript
WSQ - React for Beginners - Build Your First Web App
WSQ - Design and Build Responsive Websites from Scratch

Original slide 207

Support
If you have any enquiries during and after the class, you can contact us below

Email: enquiry@tertiaryinfotech.com
Tel: 61000613

Original slide 208

Thank You!

Sources and reproducibility

https://www.tertiarycourses.com.sg/wsq-develop-blockchain-and-web3-app-with-vibe-coding.html

https://docs.soliditylang.org/en/latest/security-considerations.html

https://web3py.readthedocs.io/en/stable/providers.html

https://eips.ethereum.org/EIPS/eip-20

https://eips.ethereum.org/EIPS/eip-721

https://ethereum.org/en/developers/docs/networks/

https://github.com/OpenZeppelin/openzeppelin-contracts/releases/tag/v5.4.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

https://alfredang.github.io/certifynft/ | source commit 2218031e7a4cb602b500ee516ccacf13b12dba27

https://alfredang.github.io/supplyverify/ | source commit 017c1105cff5a5c7e987a1ba27aee3694bf37595

https://alfredang.github.io/hashgenerator/ | source commit f498c206f5ae01fc00f2a53057893d9bbd427a8d

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg